



铁头山羊STM32教程

[I2C]

4.5. 读数据

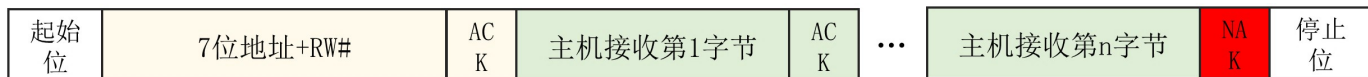
版权归东莞市明玉科技有限公司所有

未经允许不得传播

```
int My_I2C_ReceiveBytes(I2C_TypeDef *I2Cx, // I2C接口的名称
                        uint8_t  Addr, // 从机地址, 靠左
                        uint8_t  *pBuffer, // 接收缓冲区
                        uint16_t  Size); // 要发送的数据的数量 (字节)
```

作用：通过I2C从从机读取若干个字节

返回：0 - 读取成功 -1 - 寻址失败



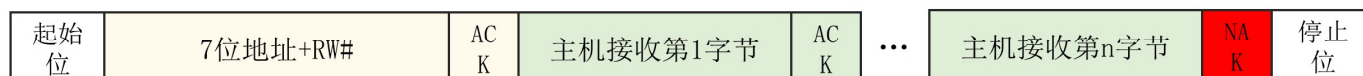
版权归东莞市明玉科技有限公司所有

非经允许不得传播

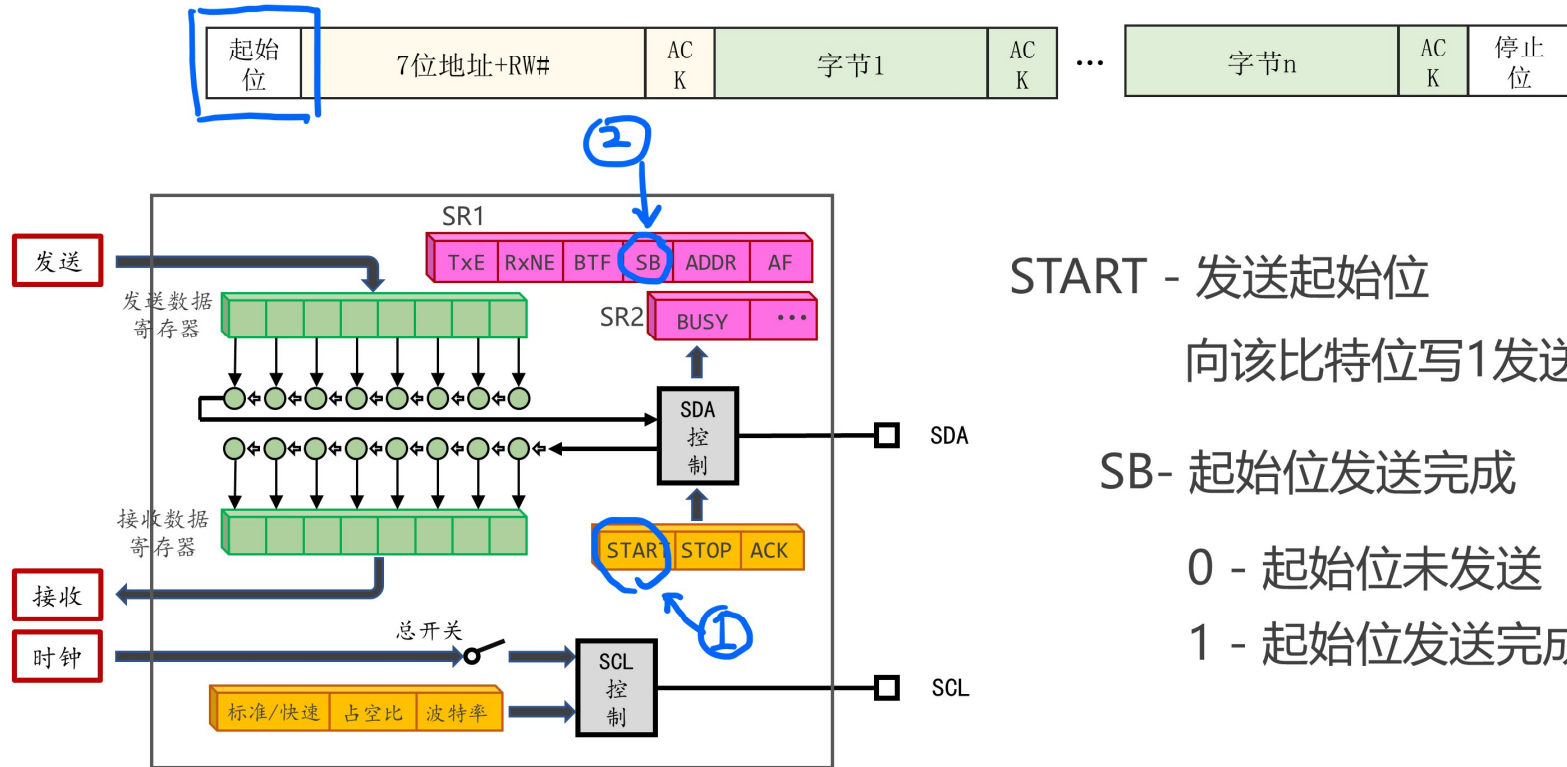
- 1. 数据读取的流程
- 2. 发送起始位和地址
- 3. 如何发送ACK和NAK
- 4. 如何发送停止位
- 5. Size=1
- 6. Size=2
- 7. Size>2

版权归东莞市明玉科技有限公司所有

未经允许不得传播



版权归东莞市明玉科技有限公司所有
未经允许不得传播



START - 发送起始位

向该比特位写1发送起始位

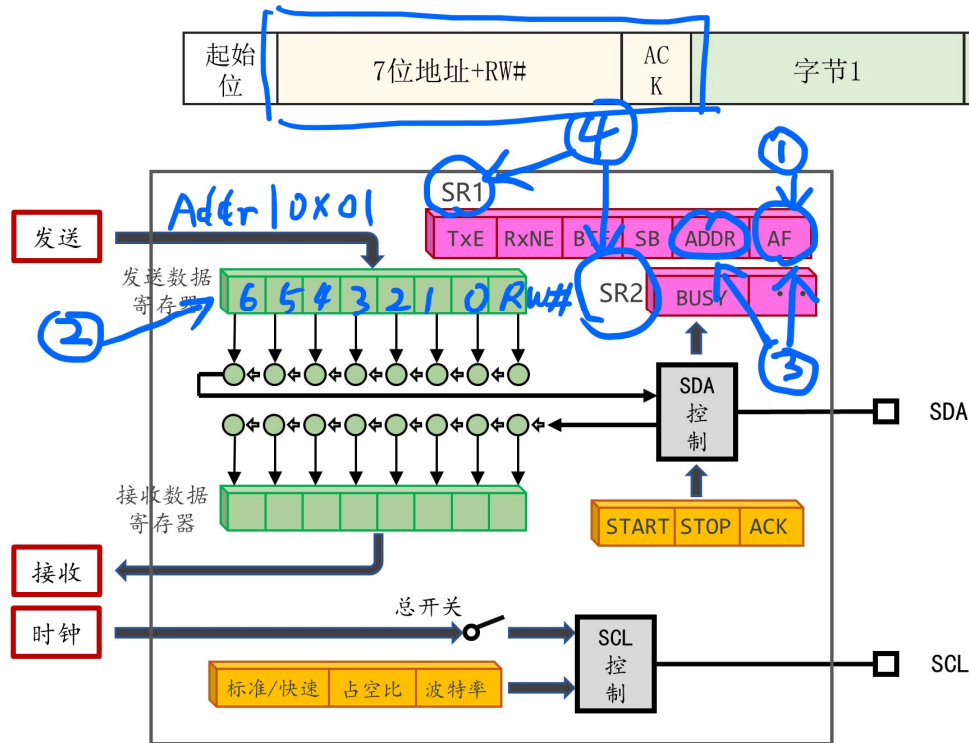
SB- 起始位发送完成

0 - 起始位未发送

1 - 起始位发送完成

版权归东莞市明玉科技有限公司所有

未经允许不得传播



AF - Acknowledge Failure 应答失败

1 - 未收到ACK

ADDR - 寻址成功

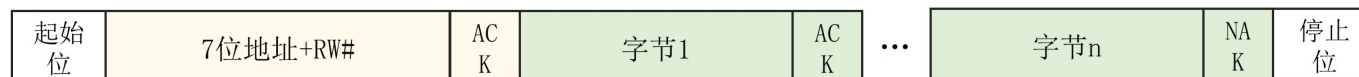
1 - 寻址成功

版权归东莞市明玉科技有限公司所有

未经允许不得传播

Addr	A6	A5	A4	A3	A2	A1	A0	x
0x01	0	0	0	0	0	0	0	1
	A6	A5	A4	A3	A2	A1	A0	1

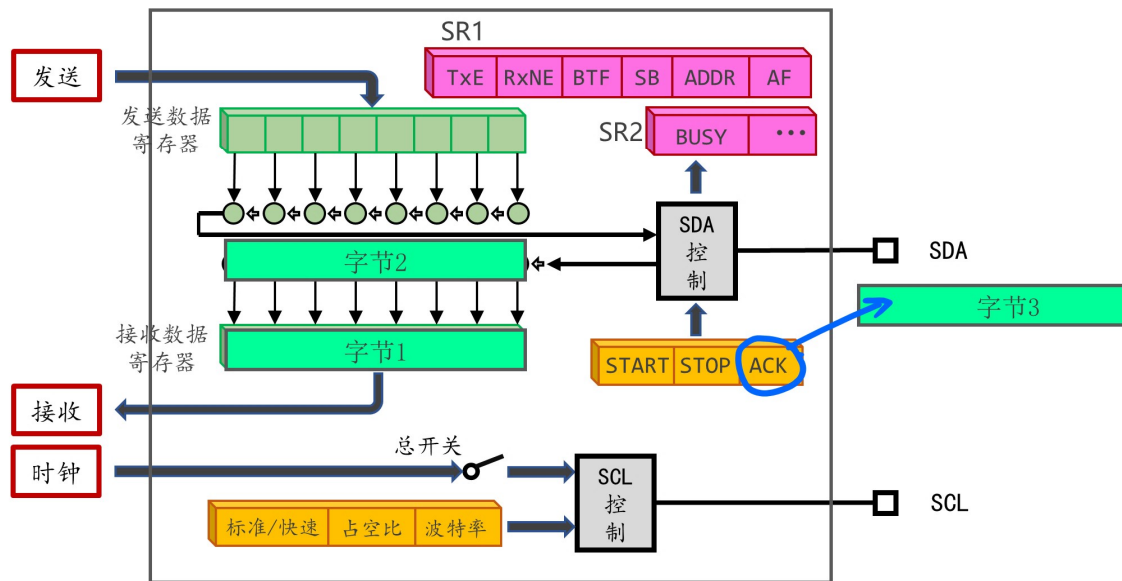
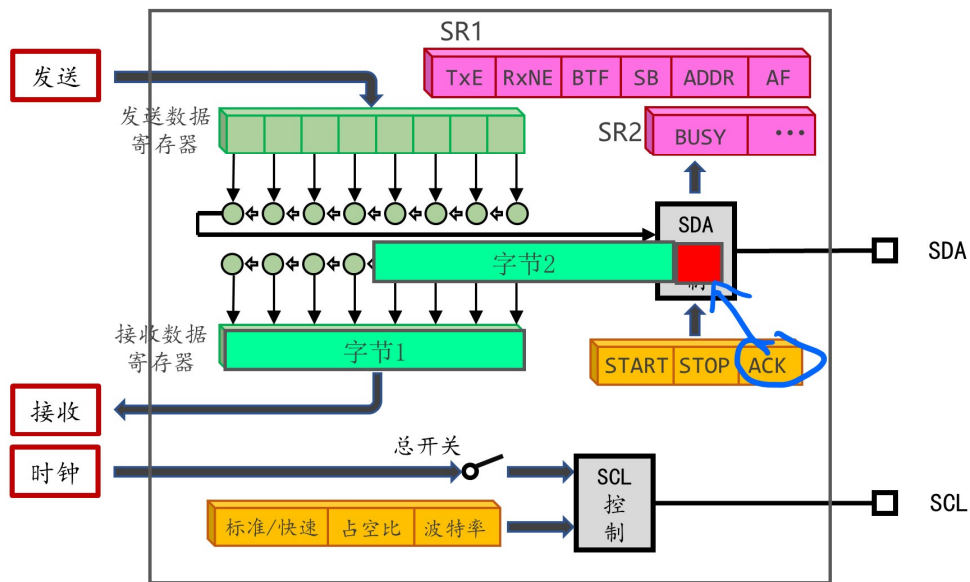
版权归东莞市明玉科技有限公司所有
未经允许不得传播



版权归东莞市明玉科技有限公司所有
未经允许不得传播

0 - 发送NAK 1 - 发送ACK

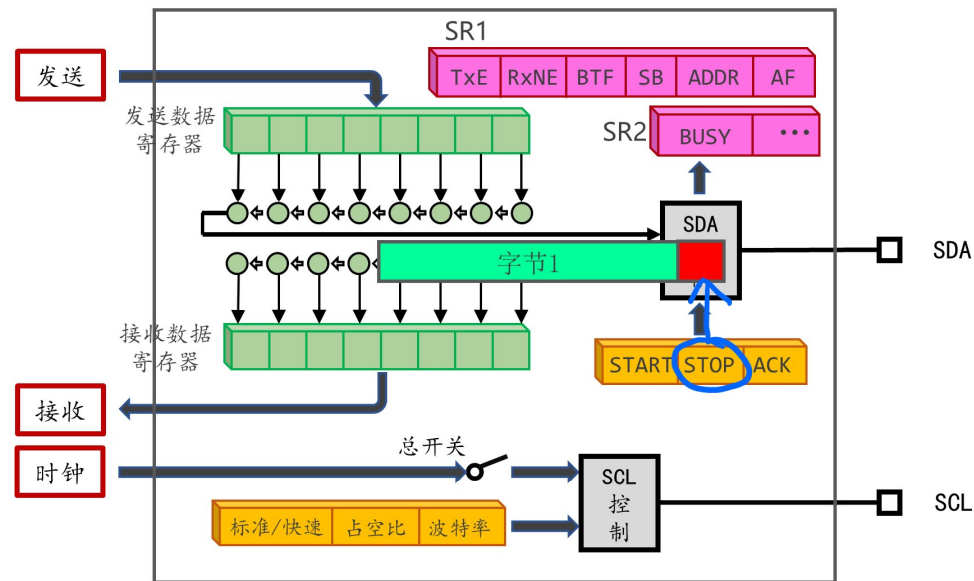
只作用于正在被接收的字节



版权归东莞市明玉科技有限公司所有

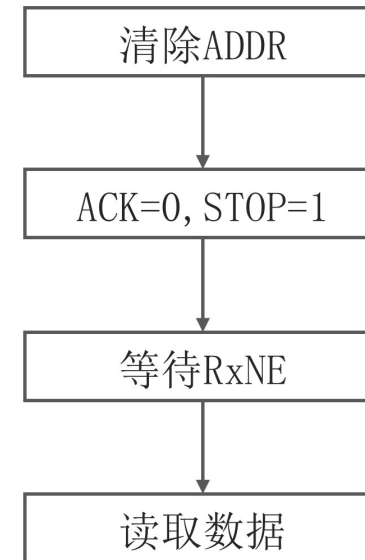
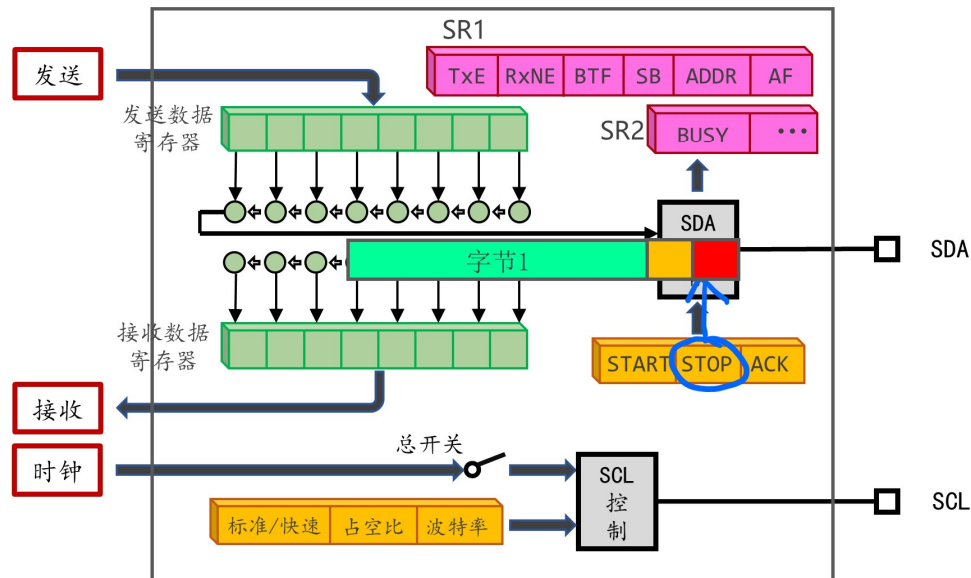
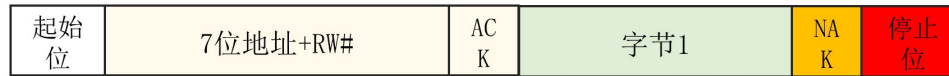
未经允许不得传播

STOP - 写1发送停止位（当前字节接收完成之后）

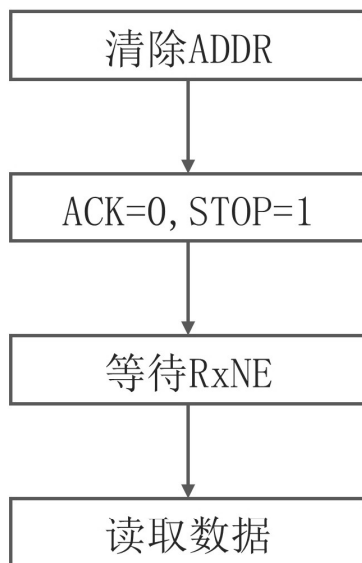


STOP作用于正在被接收的字节之后

版权归东莞市明玉科技有限公司所有
未经允许不得传播



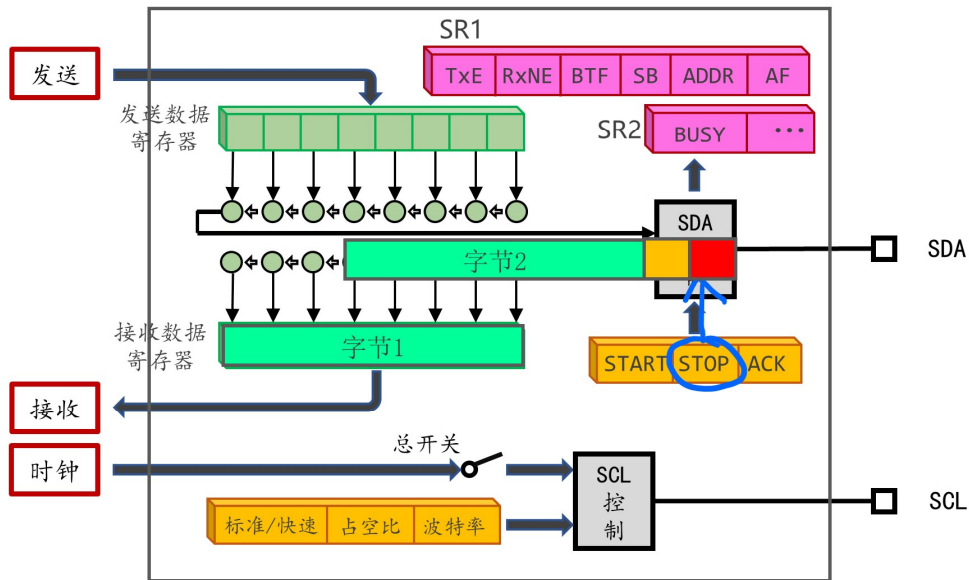
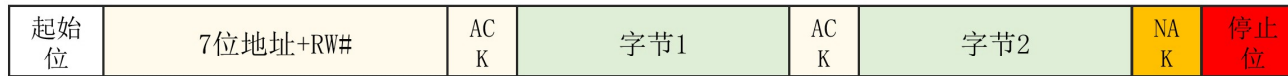
版权归东莞市明玉科技有限公司所有
非经允许不得传播



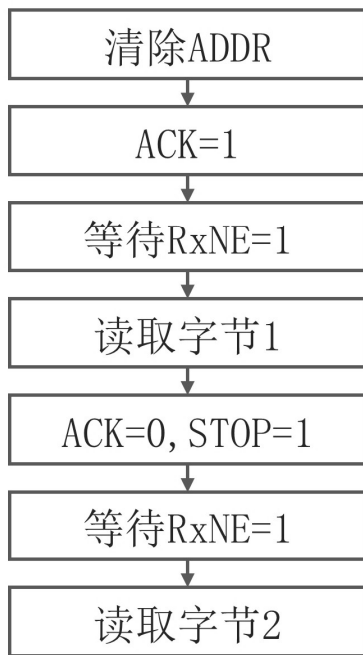
```
if(Size == 1){  
    // 清除ADDR  
    I2C_ReadRegister(I2Cx, I2C_Register_SR1);  
    I2C_ReadRegister(I2Cx, I2C_Register_SR2);  
    // 向ACK写0  
    I2C_AcknowledgeConfig(I2Cx, DISABLE);  
    // 发送停止位  
    I2C_GenerateSTOP(I2Cx, ENABLE);  
    // 等待RxNE置位  
    while(I2C_GetFlagStatus(I2Cx, I2C_FLAG_RXNE) == RESET);  
    // 读取数据  
    pBuffer[0] = I2C_ReceiveData(I2Cx);  
}
```

版权归东莞市明玉科技有限公司所有

非经允许不得传播



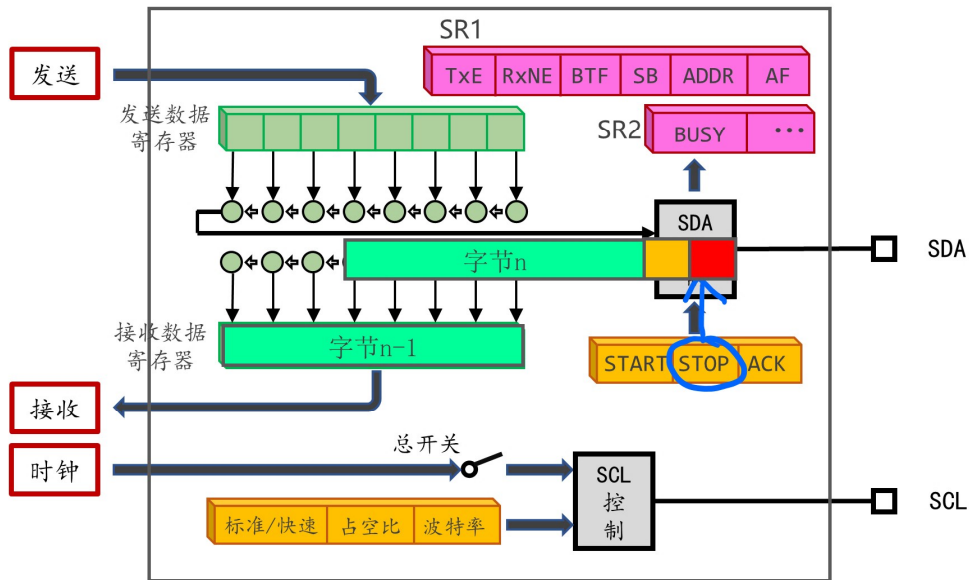
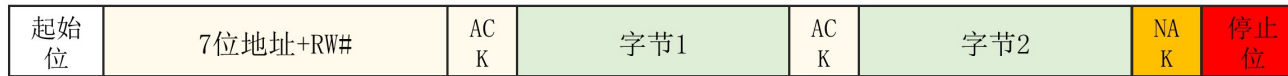
版权归东莞市明玉科技有限公司所有
未经允许不得传播



```
if(Size == 2){  
    // 清除ADDR  
    I2C_ReadRegister(I2Cx, I2C_Register_SR1);  
    I2C_ReadRegister(I2Cx, I2C_Register_SR2);  
    // 向ACK写1  
    I2C_AcknowledgeConfig(I2Cx, ENABLE);  
    // 等待RxNE置位  
    while(I2C_GetFlagStatus(I2Cx, I2C_FLAG_RXNE) == RESET);  
    // 读取字节1  
    *pData[0] = I2C_ReceiveData(I2Cx);  
    // 向ACK写0  
    I2C_AcknowledgeConfig(I2Cx, DISABLE);  
    // 发送停止位  
    I2C_GenerateStop(I2Cx, ENABLE);  
    // 等待RxNE置位  
    while(I2C_GetFlagStatus(I2Cx, I2C_FLAG_RXNE) == RESET);  
    *pData[1] = I2C_ReceiveData(I2Cx);  
}
```

版权归东莞市明玉科技有限公司所有

未经允许不得传播



重复N-1次

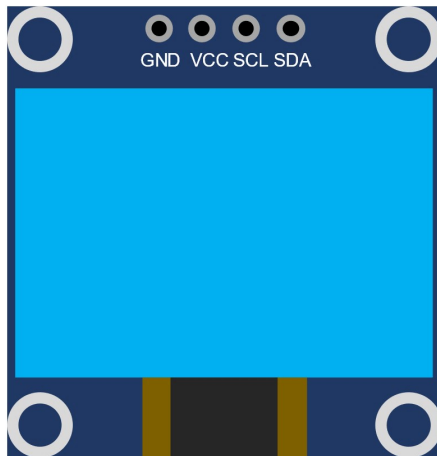
版权归东莞市明玉科技有限公司所有
未经允许不得传播

Table 9-2 : Read Command Table

Bit Pattern	Command	Description
D ₇ D ₆ D ₅ D ₄ D ₃ D ₂ D ₁ D ₀	Status Register Read	D[7]: Reserved D[6]: “1” for display OFF / “0” for display ON D[5]: Reserved D[4]: Reserved D[3]: Reserved D[2]: Reserved D[1]: Reserved D[0]: Reserved

$D_6 = 0$, 表示屏幕开启

$D_6 = 1$, 表示屏幕关闭



```
uint8_t rcvd;  
My_I2C_ReceiveBytes(I2C1, 0x78,&rcvd,1);  
if((rcvd & (0x01<<6))==0){ // 检测到屏亮  
    GPIO_WriteBit(GPIOC, GPIO_Pin_13, Bit_RESET); // 亮  
}else{ // 检测到屏灭  
    GPIO_WriteBit(GPIOC, GPIO_Pin_13, Bit_SET); // 灭灯  
}
```

版权归东莞市明玉科技有限公司所有

未经允许不得传播